

Agent Trajectory Debugger — Complete Build Plan

Build India Hackathon | Feb 15, 2026 | 7 Hours | Solo

1. WHAT IS THIS?

An open-source observability and debugging tool for AI coding agents. It intercepts every decision an AI agent makes (tool calls, file edits, bash commands, API calls), builds a real-time trajectory graph, detects failure patterns (loops, contradictions, context rot), and lets you visually pinpoint exactly where and why an agent session went off the rails.

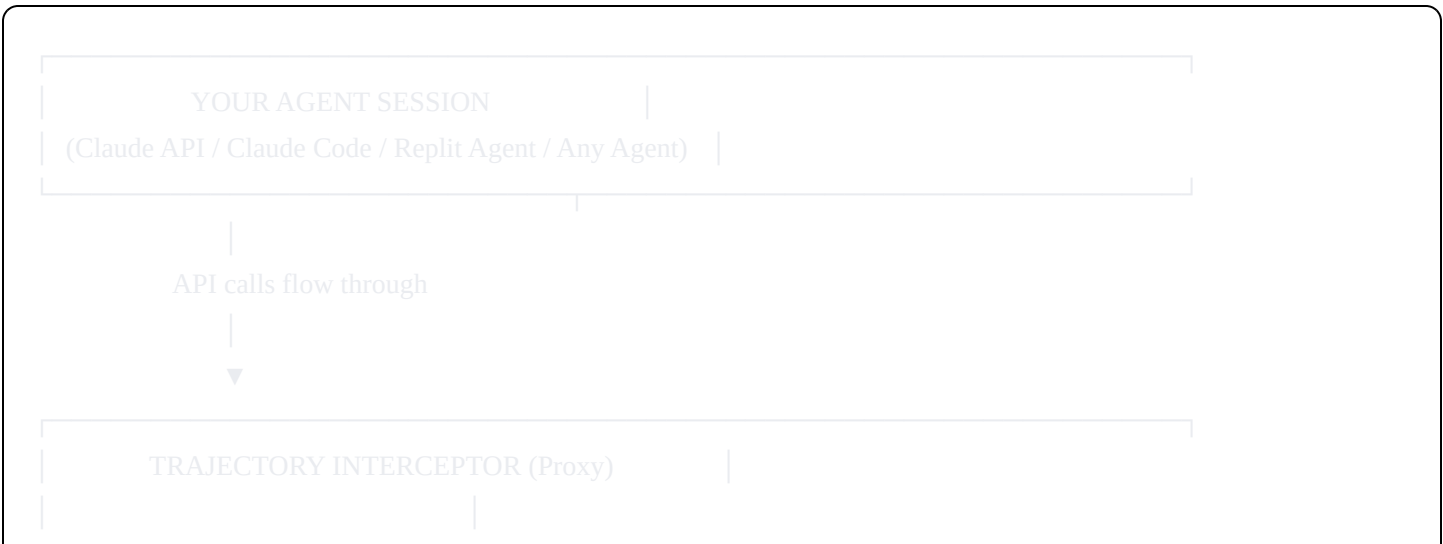
One-liner pitch: "Datadog for AI Agents — see exactly when and why your agent broke."

The problem it solves (globally):

- Replit blogged in Jan 2026 that long agent trajectories compound failures and static prompts can't fix it
- The "Everything Claude Code" hackathon winner identified "context rot" as the #1 pain point — around hour 3, Claude starts forgetting, contradicting, and looping
- Every company deploying AI agents (coding or otherwise) has zero observability into WHY the agent failed
- Today, when an agent goes wrong, you stare at a chat log. There's no structured way to replay, analyze, or branch from a failure point

What it is NOT: It is NOT a chatbot wrapper, NOT a prompt playground, NOT a UI around Claude. It is infrastructure tooling.

2. HOW IT WORKS — ARCHITECTURE





Three components:

Component 1: Trajectory Interceptor (Python SDK wrapper)

- Drop-in replacement for `anthropic.Anthropic()` client
- Monkey-patches or wraps the `messages.create()` method
- For every API call, captures and logs:
 - Full request (messages, tools, system prompt)
 - Full response (content, tool_use blocks, stop_reason)
 - Token counts (input_tokens, output_tokens, cache stats)
 - Cumulative context utilization %
 - Wall-clock time per call
 - Tool names invoked
- Writes structured JSONL to a log file + exposes via a local websocket for live streaming

Component 2: Trajectory Analyzer (Python)

- Reads the JSONL trajectory log
- Computes trajectory health metrics:
 - **Context Decay Rate:** How fast is context being consumed? (tokens/step)
 - **Repetition Score:** Are the same tool calls / file reads happening repeatedly?
 - **Decision Coherence:** Is the agent contradicting previous decisions?
 - **Loop Detection:** Is the agent stuck in a fix → break → fix cycle?
 - **Tool Diversity:** Is the agent exploring or tunnel-visioning?
 - **Error Recovery Rate:** When errors happen, does the agent recover or spiral?
- Flags anomalies: "Step 14 is where context rot began" / "Steps 22-28 form a loop"
- Uses Claude API itself for meta-analysis: sends trajectory summary to Claude asking "analyze this agent session and identify failure points"

Component 3: Web Dashboard (React on Replit)

- Deployed on Replit to showcase Replit integration
 - Reads trajectory data (uploaded JSON or live websocket)
 - Renders:
 - **Timeline View:** Horizontal scrollable timeline, each node = one agent step, color-coded by health (green → yellow → red)
 - **Health Gauge:** Overall trajectory health score (0-100)
 - **Token Budget Chart:** Area chart showing context window fill over time
 - **Tool Call Breakdown:** Bar chart of which tools were used and how often
 - **Failure Markers:** Red pins on the timeline where anomalies detected
 - **Step Detail Panel:** Click any step to see full request/response, token counts, what changed
 - **Trajectory Diff:** Compare what the agent "said it would do" vs what it actually did
-

3. CORE FEATURES — MVP FOR SUBMISSION

These MUST be working before you submit:

Feature 1: Instrumented SDK Wrapper (THE CORE)

```
python
```

```
from agent_debugger import InstrumentedClient

# Drop-in replacement — just change the import
client = InstrumentedClient(api_key="sk-...")

# Use exactly like normal Anthropic SDK
response = client.messages.create(
    model="claude-sonnet-4-5-20250929",
    max_tokens=4096,
    tools=[...],
    messages=[...]
)

# Everything is automatically logged to trajectory.jsonl
```

What it captures per step:

json

```

{
  "step_id": 14,
  "timestamp": "2026-02-15T14:23:01Z",
  "request": {
    "message_count": 28,
    "total_input_tokens": 45230,
    "tools_provided": ["read_file", "write_file", "bash", "grep"],
    "system_prompt_tokens": 1200
  },
  "response": {
    "output_tokens": 890,
    "stop_reason": "tool_use",
    "tools_called": [
      {"name": "read_file", "input": {"path": "src/app.py"}},
      {"name": "write_file", "input": {"path": "src/app.py"}}
    ],
    "text_content_summary": "Modifying the database connection handler..."
  },
  "trajectory_metrics": {
    "context_utilization_pct": 67.2,
    "cumulative_tokens": 142800,
    "context_window_max": 200000,
    "tokens_since_last_compact": 89400,
    "step_duration_ms": 3200,
    "files_touched": ["src/app.py"],
    "unique_tools_this_session": 4,
    "repeated_file_reads": 3
  }
}

```

Feature 2: Anomaly Detection Engine

Five detection algorithms, each simple but effective:

a) Loop Detector

- Tracks sequence of (tool_name, target_file) pairs
- If the same sequence of 3+ tool calls repeats within a sliding window of 10 steps → flag as loop
- Example: read_file(app.py) → write_file(app.py) → bash(npm test) → read_file(app.py) → write_file(app.py) → bash(npm test) = LOOP

b) Context Decay Tracker

- Monitors context_utilization_pct over time
- Computes rate of change (tokens consumed per step)
- Flags when: utilization > 70% AND rate is accelerating (agent is burning context faster)

c) Repetition Scorer

- Hashes the tool call signatures (tool_name + first 100 chars of input)
- Tracks frequency of each hash
- If any single hash appears > 3x in last 15 steps → flag as repetitive

d) Error Spiral Detector

- Tracks sequences where: tool_result contains error → agent tries fix → tool_result contains error again
- If error count increases monotonically over 4+ steps → flag as error spiral

e) Contradiction Detector (uses Claude)

- At analysis time, sends pairs of agent text outputs from different steps to Claude
- Asks: "Does step N contradict the plan/decision stated in step M?"
- This is the "meta-analysis" layer — Claude debugging Claude

Feature 3: Web Dashboard

Must have these views working:

Timeline View (Primary)

- Horizontal scrollable bar
- Each step = a node (circle or rectangle)
- Color: green (healthy) → yellow (warning) → red (anomaly)
- Click a node → shows step detail in side panel
- Anomaly steps have a pulsing red marker with label ("LOOP DETECTED", "CONTEXT ROT", etc.)

Metrics Panel

- Overall Health Score: 0-100 (weighted average of all metrics)
- Context Budget: area chart (x = step, y = tokens used / tokens available)
- Tool Distribution: horizontal bar chart
- Error Rate: line chart over steps

Step Detail Panel

- Shows: what the agent saw (summarized input), what it decided (output text), what tools it called, token cost of this step, any anomalies flagged
-

4. TECH STACK

INTERCEPTOR + ANALYZER:

- └─ Python 3.11+
- └─ anthropic SDK (the real one, which you wrap)
- └─ dataclasses + json for log schema
- └─ hashlib for tool call hashing (loop/repetition detection)
- └─ websockets (for live streaming to dashboard)

WEB DASHBOARD:

- └─ React 18+ (single-page app)
- └─ Tailwind CSS (styling)
- └─ Recharts (charts: area, bar, line)
- └─ Deployed on Replit
- └─ Reads trajectory JSONL (file upload + websocket for live)

DEMO AGENT:

- └─ Python script using your InstrumentedClient
- └─ Simulates a coding agent that goes wrong
- └─ Produces a trajectory.jsonl that the dashboard consumes

Why this stack?

- Python interceptor: matches the Anthropic SDK language, drop-in replacement story
 - React dashboard on Replit: directly shows Replit usage, judges can see the deployed app
 - No database needed: JSONL files are the data store (keeps it simple for 7 hours)
 - Recharts: you know React, Recharts is the fastest way to get clean charts
-

5. WHAT THE DEMO LOOKS LIKE

This is what you show the judges in your 3-minute demo:

Act 1: "Here's what agents do today" (30 seconds)

- Show a normal Claude API coding session running
- It starts great, then around step 15 it starts looping, re-reading files, contradicting itself
- "Today, you have NO idea why this happened. You just see it broke."

Act 2: "Now watch with the Trajectory Debugger" (90 seconds)

- Same session, but instrumented with your SDK
- Open the web dashboard
- Show the timeline — green nodes transitioning to yellow to red
- Click on the first red node: "HERE. Step 14. This is where context utilization hit 72% and the agent started re-reading files it already read"
- Show the loop detection: "Steps 22-28 are a fix-break-fix loop on the same file"
- Show the context budget chart: "You can see the exact moment context started running out"
- Show the health score dropping from 92 to 34

Act 3: "This works for ANY agent" (30 seconds)

- "This isn't Claude-specific. The interceptor pattern works for any LLM API"
- "This is the observability layer the entire agent ecosystem is missing"
- "Replit Agent, Claude Code, custom agents — they all need this"

Act 4: "The pitch" (30 seconds)

- "Every company deploying AI agents will need trajectory debugging"
- "This is Datadog for the agent era"
- "Open source interceptor, paid dashboard and analytics — classic devtool business model"

6. HOUR-BY-HOUR BUILD PLAN

Hour 0-1: Interceptor Core (THE MOST CRITICAL HOUR)

Goal: Working SDK wrapper that logs everything to JSONL

Build `interceptor.py`:

```
python
```



```

import anthropic
import json
import time
from dataclasses import dataclass, asdict
from datetime import datetime

class InstrumentedClient:
    def __init__(self, **kwargs):
        self._client = anthropic.Anthropic(**kwargs)
        self._log_file = kwargs.get("trajectory_log", "trajectory.jsonl")
        self._step_counter = 0
        self._cumulative_input_tokens = 0
        self._cumulative_output_tokens = 0
        self._tool_call_history = []
        self._file = open(self._log_file, "a")
        self.messages = self # so client.messages.create() works

    def create(self, **kwargs):
        self._step_counter += 1
        start_time = time.time()

        # Call the real API
        response = self._client.messages.create(**kwargs)

        duration_ms = (time.time() - start_time) * 1000

        # Extract metrics
        input_tokens = response.usage.input_tokens
        output_tokens = response.usage.output_tokens
        self._cumulative_input_tokens += input_tokens
        self._cumulative_output_tokens += output_tokens

        # Extract tool calls
        tool_calls = []
        text_content = ""
        for block in response.content:
            if block.type == "tool_use":
                tool_calls.append({
                    "name": block.name,
                    "input_summary": str(block.input)[:200]
                })
            self._tool_call_history.append(
                f"{block.name}:{str(block.input)[:100]}"

```

```

    )
    elif block.type == "text":
        text_content = block.text[:300]

# Build log entry
entry = {
    "step_id": self._step_counter,
    "timestamp": datetime.utcnow().isoformat(),
    "input_tokens": input_tokens,
    "output_tokens": output_tokens,
    "cumulative_tokens": self._cumulative_input_tokens + self._cumulative_output_tokens,
    "context_utilization_pct": round(
        (self._cumulative_input_tokens / 200000) * 100, 1
    ),
    "duration_ms": round(duration_ms),
    "stop_reason": response.stop_reason,
    "tools_called": tool_calls,
    "text_summary": text_content,
    "message_count": len(kwargs.get("messages", [])),
    "model": kwargs.get("model", "unknown"),
}

# Write to log
self._file.write(json.dumps(entry) + "\n")
self._file.flush()

return response # Return the real response unchanged

```

Test with a simple 3-step agent call to confirm it works.

Deliverable: interceptor.py that wraps Anthropic SDK and produces trajectory.jsonl

Hour 1-2: Anomaly Detection Algorithms

Goal: Five detection algorithms that analyze a trajectory log

Build `analyzer.py`:

```
python
```

```

import json
from collections import Counter, deque

class TrajectoryAnalyzer:
    def __init__(self, trajectory_path):
        self.steps = []
        with open(trajectory_path) as f:
            for line in f:
                self.steps.append(json.loads(line))

    def detect_loops(self, window=10, min_repeat=2):
        """Detect repeated sequences of tool calls"""
        anomalies = []
        signatures = [
            "|".join(tc["name"] for tc in step.get("tools_called", []))
            for step in self.steps
        ]
        for i in range(len(signatures)):
            for seq_len in range(2, 5): # Check sequences of length 2-4
                if i + seq_len * min_repeat > len(signatures):
                    break
                pattern = signatures[i:i+seq_len]
                if pattern == [""] * seq_len:
                    continue
                repeats = 0
                for j in range(min_repeat):
                    if signatures[i+j*seq_len:i+(j+1)*seq_len] == pattern:
                        repeats += 1
                if repeats >= min_repeat:
                    anomalies.append({
                        "type": "LOOP",
                        "start_step": self.steps[i]["step_id"],
                        "end_step": self.steps[i + seq_len * repeats - 1]["step_id"],
                        "pattern": pattern,
                        "severity": "HIGH"
                    })
        return anomalies

    def detect_context_decay(self, warning_threshold=60, critical_threshold=80):
        """Flag when context utilization is growing too fast"""
        anomalies = []
        for i in range(1, len(self.steps)):
            curr = self.steps[i]["context_utilization_pct"]

```

```

prev = self.steps[i-1]["context_utilization_pct"]
rate = curr - prev
if curr > critical_threshold:
    anomalies.append({
        "type": "CONTEXT_ROT",
        "step_id": self.steps[i]["step_id"],
        "utilization": curr,
        "rate": round(rate, 2),
        "severity": "CRITICAL" if curr > 85 else "HIGH"
    })
elif curr > warning_threshold and rate > 5:
    anomalies.append({
        "type": "CONTEXT_DECAY",
        "step_id": self.steps[i]["step_id"],
        "utilization": curr,
        "rate": round(rate, 2),
        "severity": "MEDIUM"
    })
return anomalies

```

```

def detect_repetition(self, threshold=3, window=15):
    """Detect repeated identical tool calls"""
    anomalies = []
    recent = deque(maxlen=window)
    for step in self.steps:
        for tc in step.get("tools_called", []):
            sig = f"{tc['name']}:{tc.get('input_summary', '')[:80]}"
            recent.append(sig)
            count = sum(1 for s in recent if s == sig)
            if count >= threshold:
                anomalies.append({
                    "type": "REPETITION",
                    "step_id": step["step_id"],
                    "tool": tc["name"],
                    "count": count,
                    "severity": "MEDIUM"
                })
    return anomalies

```

```

def detect_error_spiral(self):
    """Detect increasing error sequences"""
    anomalies = []
    consecutive_errors = 0
    spiral_start = None

```

```

for step in self.steps:
    has_error = step.get("stop_reason") == "error" or \
        "error" in step.get("text_summary", "").lower()
    if has_error:
        consecutive_errors += 1
        if consecutive_errors == 1:
            spiral_start = step["step_id"]
        if consecutive_errors >= 3:
            anomalies.append({
                "type": "ERROR_SPIRAL",
                "start_step": spiral_start,
                "current_step": step["step_id"],
                "consecutive_errors": consecutive_errors,
                "severity": "HIGH"
            })
    else:
        consecutive_errors = 0
        spiral_start = None
return anomalies

def compute_health_score(self):
    """Overall trajectory health 0-100"""
    all_anomalies = (
        self.detect_loops() +
        self.detect_context_decay() +
        self.detect_repetition() +
        self.detect_error_spiral()
    )
    severity_weights = {"CRITICAL": 20, "HIGH": 10, "MEDIUM": 5}
    penalty = sum(severity_weights.get(a["severity"], 0) for a in all_anomalies)
    return max(0, 100 - penalty)

def full_analysis(self):
    """Return complete analysis for the dashboard"""
    return {
        "total_steps": len(self.steps),
        "health_score": self.compute_health_score(),
        "anomalies": {
            "loops": self.detect_loops(),
            "context_decay": self.detect_context_decay(),
            "repetition": self.detect_repetition(),
            "error_spirals": self.detect_error_spiral(),
        },
        "steps": self.steps,
    }

```

```

"summary": {
    "total_tokens": self.steps[-1]["cumulative_tokens"] if self.steps else 0,
    "final_context_pct": self.steps[-1]["context_utilization_pct"] if self.steps else 0,
    "total_tool_calls": sum(
        len(s.get("tools_called", [])) for s in self.steps
    ),
    "total_duration_s": sum(
        s.get("duration_ms", 0) for s in self.steps
    ) / 1000,
}
}

```

Deliverable: analyzer.py that reads trajectory.jsonl and outputs full analysis JSON

Hour 2-3: Demo Agent (Generates a Compelling Trajectory)

Goal: Script that simulates a realistic agent session that goes wrong

This is CRITICAL for the demo. You need a trajectory.jsonl that tells a compelling story.

Build `demo_agent.py`:

```

python
"""
Simulates a coding agent that:
- Starts well (steps 1-10): reads files, plans, makes good edits
- Degrades (steps 11-18): starts re-reading files, context grows
- Fails (steps 19-25): enters a fix loop, contradicts earlier decisions
- Dies (steps 26-30): context nearly full, repeating same operations
"""

```

Two approaches (pick based on API access at hackathon):

Approach A (Preferred): Real agent session If you have Claude API access, write a real coding agent:

- Give it a task: "Build a REST API with auth, database, and tests in Python"
- Use your InstrumentedClient
- Let it run for 25-30 steps
- The natural failures will happen on their own

Approach B (Fallback): Synthetic trajectory If API is slow/limited, generate a realistic trajectory.jsonl manually:

- 30 steps with realistic token counts
- Steps 1-10: healthy (low utilization, diverse tools, no repeats)
- Steps 11-18: degrading (rising utilization, some file re-reads)
- Steps 19-25: failing (loops, error spirals)
- Steps 26-30: dead (>90% utilization, pure repetition)

I recommend doing BOTH: run a real session during Hour 2, AND have a pre-built synthetic trajectory as backup.

Deliverable: trajectory.jsonl with 25-30 steps showing a clear degradation story

Hour 3-5: Web Dashboard (React on Replit)

Goal: Beautiful, functional dashboard that visualizes the trajectory

This is where you spend the most time. The dashboard IS the demo.

File structure:

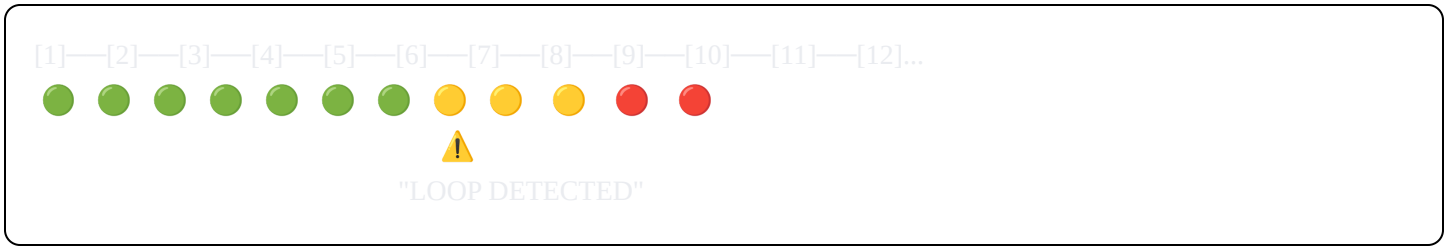
```
dashboard/
├── src/
│   ├── App.jsx      # Main layout
│   ├── TimelineView.jsx # The hero component
│   ├── MetricsPanel.jsx # Health score + charts
│   ├── StepDetail.jsx  # Side panel for step inspection
│   ├── AnomalyBadge.jsx # Red badges on timeline
│   └── data/
│       └── sample.json # Pre-loaded trajectory analysis
├── package.json
└── index.html
```

Design direction: Dark theme, "mission control" aesthetic

- Dark background (● #0a0a0f)
- Accent colors: Green (● #22c55e) for healthy, Amber (● #f59e0b) for warning, Red (● #ef4444) for critical
- Monospace font for data (JetBrains Mono or Fira Code from Google Fonts)
- Sans-serif for labels (DM Sans or Space Grotesk)
- Subtle grid/dot pattern background (like a radar screen)
- Glow effects on anomaly markers

Key components to build:

a) TimelineView (Most important visual)



- Horizontal scrollable
- Each node clickable
- Anomaly nodes have pulsing animation + label
- Color = step health (derived from metrics)

b) Context Budget Chart (Recharts AreaChart)

- X axis: step number
- Y axis: context utilization %
- Area fill: green → yellow → red gradient
- Dashed line at 80% (danger zone)
- Annotations at anomaly points

c) Health Score Gauge

- Big circular gauge (like a speedometer)
- 0-100 score
- Color changes with score
- Shows "HEALTHY" / "DEGRADED" / "FAILING" / "CRITICAL"

d) Tool Distribution (Recharts BarChart)

- Horizontal bars
- Each tool name + count
- Highlighted if a tool has repetition anomalies

e) Step Detail Panel (slides in from right)

- Shows when you click a timeline node
- Displays: step_id, timestamp, tokens used, tools called, text summary

- If step has anomaly, shows the anomaly detail with red border

Deliverable: Deployed React app on Replit showing the trajectory dashboard

Hour 5-6: Claude Meta-Analysis Integration

Goal: Use Claude to analyze the trajectory and generate human-readable insights

This is the "wow factor" — Claude debugging Claude.

Build `meta_analyzer.py`:

```
python
```

```

import anthropic
import json

def analyze_trajectory_with_claude(trajectory_data):
    client = anthropic.Anthropic()

    # Summarize trajectory for Claude (don't send raw data, too many tokens)
    summary = {
        "total_steps": trajectory_data["total_steps"],
        "health_score": trajectory_data["health_score"],
        "anomalies_found": len([
            a for category in trajectory_data["anomalies"].values()
            for a in category
        ]),
        "step_summaries": [
            {
                "step": s["step_id"],
                "tools": [t["name"] for t in s.get("tools_called", [])],
                "text": s.get("text_summary", "")[:150],
                "context_pct": s["context_utilization_pct"],
            }
            for s in trajectory_data["steps"]
        ]
    }

    response = client.messages.create(
        model="claude-sonnet-4-5-20250929",
        max_tokens=1500,
        messages=[{
            "role": "user",
            "content": f"""You are an AI agent debugging expert. Analyze this
agent trajectory and identify:
1. The exact step where performance started degrading and WHY
2. The root cause of any loops or repeated operations
3. What the agent SHOULD have done differently
4. A severity assessment (CRITICAL / HIGH / MEDIUM / LOW)

Trajectory data:
{json.dumps(summary, indent=2)}

Respond in JSON with keys: degradation_point, root_cause,
recommendation, severity, narrative (2-3 sentence human-readable summary)"""
        }]
    )

```

```
)  
return response.content[0].text
```

Add the Claude-generated narrative to the dashboard as a "AI Analysis" card at the top.

Deliverable: Claude-powered narrative analysis displayed on dashboard

Hour 6-6.5: Integration + Polish

Goal: Everything works end-to-end

- ☐ Run demo_agent.py → produces trajectory.jsonl
 - ☐ Run analyzer.py on trajectory.jsonl → produces analysis.json
 - ☐ Run meta_analyzer.py → produces Claude narrative
 - ☐ Load analysis.json + narrative into dashboard
 - ☐ Verify all charts render correctly
 - ☐ Verify timeline clicking works
 - ☐ Verify anomaly markers show
 - ☐ Add loading states
 - ☐ Test on mobile viewport (judges may view on phones)
 - ☐ Deploy dashboard on Replit (public URL)
-

Hour 6.5-7: Demo Prep + README

Goal: Pitch-ready

- ☐ Write README.md with:
 - What it is (one paragraph)
 - Architecture diagram (ASCII or image)
 - Quick start: `pip install agent-debugger` (aspirational)
 - Screenshot of dashboard
 - "Built at Build India 2026 with Claude + Replit"
- ☐ Practice demo flow (3 minutes):
 1. Show the problem (agent going wrong with no visibility)
 2. Show the interceptor (2 lines of code to instrument)
 3. Show the dashboard (timeline, health score, anomalies)
 4. Show Claude meta-analysis ("Claude debugging Claude")
 5. The pitch ("Datadog for AI Agents")

☐ Prepare backup: have a pre-recorded GIF of the dashboard in case live demo fails

7. WHAT TO SKIP (CRITICAL FOR TIME MANAGEMENT)

Do NOT build:

- ❌ User authentication
- ❌ Database storage (use flat JSON files)
- ❌ Real-time websocket streaming (use file upload to dashboard)
- ❌ Multi-session comparison
- ❌ Branching / replay functionality (mention it as future work)
- ❌ Support for non-Anthropic APIs (mention as roadmap)
- ❌ pip package / proper packaging
- ❌ Tests (it's a hackathon)
- ❌ Error handling edge cases

Do NOT over-polish:

- ❌ Don't spend more than 10 min on the README
 - ❌ Don't spend more than 15 min on the demo script
 - ❌ Don't add features after Hour 6 — only fix bugs
-

8. RISK MITIGATION

Risk	Mitigation
Claude API rate limits / slow	Have pre-generated trajectory.jsonl as backup
Replit deployment issues	Have local dev server as backup; deploy early (Hour 4)
Dashboard takes too long	Start with timeline + one chart, add others incrementally
Anomaly detection too complex	Start with just loop detection + context decay (easiest two)
Demo agent doesn't produce good failures	Use synthetic trajectory that tells a perfect story
Judges don't understand the problem	Open with "Have you ever had an AI agent go wrong and had no idea why?"

9. JUDGING OPTIMIZATION

What Anthropic judges (Beth Robertson, Irina Ghose, Yeop Lee, Danny Delaney, Rebecca Harbeck) will look for:

- Deep Claude API usage (you have: interceptor + meta-analysis using Claude)
- Technical depth (you have: five anomaly detection algorithms)
- Something that makes Claude better (you have: literally debugging Claude sessions)

What Lightspeed judges will look for:

- Global TAM (every company deploying agents = your customer)
- Business model clarity ("open-source SDK, paid dashboard + analytics")
- Technical moat (interceptor pattern + anomaly detection IP)

What Replit judges will look for:

- Built/deployed on Replit (dashboard is on Replit)
- Solves a real problem for their ecosystem (agent trajectory failures)

Name the project something memorable: Options: "AgentScope" / "TraceAgent" / "Trajectory" / "AgentPulse" / "Pathfinder"

I'd go with "**AgentLens**" — short, memorable, implies observability/visibility into agents.

10. POST-HACKATHON POTENTIAL (FOR THE PITCH)

Mention these briefly in the demo to show you've thought about the business:

- **Open-source SDK** (interceptor) → drives adoption
 - **Paid cloud dashboard** → recurring revenue (usage-based pricing per trajectory analyzed)
 - **Team features** → shared trajectory debugging for engineering teams
 - **Multi-provider support** → OpenAI, Google, Anthropic, open-source models
 - **CI/CD integration** → run trajectory analysis as part of agent testing pipeline
 - **Alert system** → "Your production agent's health dropped below 50"
 - **Benchmark dataset** → anonymized trajectories become training data for better agents
-

11. FINAL CHECKLIST BEFORE YOU START

- ☐ Anthropic API key ready and tested
- ☐ Replit account logged in
- ☐ Node.js + Python environment ready
- ☐ This document open on a second screen
- ☐ Pre-built synthetic trajectory.jsonl ready (generate it tonight)
- ☐ Dashboard color palette saved (dark theme hex codes)
- ☐ Recharts docs bookmarked: <https://recharts.org/en-US>
- ☐ Anthropic SDK docs bookmarked: <https://docs.anthropic.com>

START BUILDING THE SYNTHETIC TRAJECTORY TONIGHT. This is the one thing you can prep before the hackathon. Generate a 30-step trajectory.jsonl with a clear degradation story. This way, even if the API is slow during the hackathon, your dashboard will have data to show from minute one.